설비 고장을 예측하는 AI 데이터 분석

Matplotlib 기초

분포·범주·관계 시각화

posco

CONTENTS

목차

01 막대 그래프 — 범주를 비교

02 히스토그램 — 분포 확인

03 산점도 — 두 값의 관계

04 시각화 리포트 완성

posco

01

막대 그래프 — 범주를 비교

설비별·구역별·상태별 항목을 막대로 견주는 방법

분포·범주·관계 시각화

네 그래프를 엮어 데이터를 다각도로 보고 분석 리포트까지 완성

01 · 다각도로 보기

막대 · 히스토그램 · 산점도로
데이터의 세 얼굴

선 그래프가 흐름을 보여줬다면, 이번엔 막대로 비교를, 히스토그램으로 분포를, 산점도로 관계를 봄

02 · 리포트로 엮기

네 그래프를 한 화면에 모아
분석 리포트 완성

그린 그래프들을 흐름대로 엮어 결론까지 적는, 하나의 분석 리포트로 마무리

범주를 비교하고 싶을 때

이어지는 흐름엔 선, 떨어진 항목 비교엔 막대

선 그래프

이어지는 변화

요일이 지나며 온도가 변하듯, 시간 축처럼 이어지는 흐름에 어울림

예) 날짜별 평균 온도

막대 그래프

떨어진 항목 비교

A호기·B호기·C호기처럼 서로 이어지지 않은 범주 항목의 크기를 견줄 때

예) 설비별 평균 온도, 구역별 점검 건수

posco

plt.bar()로 막대 그래프 그리기

plot과 거의 같은 방식, 첫 자리에 항목 이름

CODE

```
plt.bar(['A호기','B호기','C호기'],
 [64, 70, 77])
plt.show()
```

첫 자리는 글자 항목 이름, 둘째 자리는 막대 높이 — 두 목록 개수는 반드시 같아야 함

posco

막대 색·너비 옵션

color로 막대 색을, width로 막대 굵기를 조정

COLOR · 막대 색

리스트로 막대마다 다른 색

색 하나면 모두 같은 색, 색 리스트를 넣으면 막대마다 다른 색 — 정상은 파랑, 점검은 빨강처럼 의미 있게 사용

크기 비교를 넘어 **어느 게 문제인지**까지 보여줌

WIDTH · 막대 굵기

기본 0.8, 작게 하면 가늘게

0.5처럼 작게 하면 막대가 가늘어지고 간격이 넓어짐 — 막대가 많아 빽빽하면 줄이고, 적어 휑하면 키우기

기본값 0.8 — 막대 사이에 약간 틈

posco

막대 색·너비 코드

색 리스트로 C호기만 빨강, width로 막대 가늘게

CODE

```
plt.bar(['A호기','B호기','C호기'],
 [64, 70, 77],
 color=['blue','blue','red'],
 width=0.5)
plt.show()
```

색 리스트의 개수도 막대 개수와 같아야 함

posco

가로 막대 그래프 (barh)

plt.barh() — 항목 이름이 길거나 많을 때 보기 좋음

명령 · barh

h는 horizontal, 사용법은 bar와 동일

값이 가로로 늘어나므로 단위는 **xlabel**에 넣음 — 세로 막대와 반대

plt.barh(항목, 값)

쓰임새

긴 이름·많은 항목, 순위표

1구역 3번 라인 A설비처럼 긴 이름은 세로 막대에서 겹치지만, 가로 막대에서는 또렷하게 읽힘

위아래 늘어뜨려도 여유 있음

posco

막대 라벨과 값 표시하기

긴 항목 이름은 xticks rotation, 값은 막대 위 plt.text()

XTICKS(ROTATION)

긴 항목 이름을 비스듬히

항목 이름이 겹치면 rotation 옵션으로 45도 기울임 — 날짜나 긴 설비명에 유용

```
plt.xticks(rotation=45)
```

PLT.TEXT()

막대 위에 정확한 값 표시

높이로 어림짐작하는 대신 정확한 값을 한눈에 — 보고용일 때 유용

노베이스 단계에선 가능하다는 정도만 알기

posco

막대 그래프 활용과 실수 정리

쉬워 보이지만 잘못 그리면 사람을 속이는 그래프

항목	올바른 방법
세로축 시작	반드시 0부터 — 60부터 시작하면 3% 차이가 두 배로 과장
항목 순서	값 크기 순 정렬 — 맨 앞 막대가 가장 문제인 항목
범주 비교	선 아닌 막대 — 설비별 비교는 범주 비교
항목·값 개수	반드시 같게 — 선 그래프와 같은 규칙

세로축이 0부터 시작하지 않아 차이를 과장하는 것은 데이터 왜곡의 흔한 속임수

posco

실습 1. 막대 그래프 직접 그리기

설비별 평균 온도를 막대로, 색·가로 막대로 변형

목표

범주별 값을 막대로 비교하고 색·방향을 바꿔 강조

단계

· groupby로 설비별 평균 온도를 구해 막대로 그리기

· 색 리스트로 가장 뜨거운 설비만 강조하고 너비 조절

· barh로 가로 막대로 바꾸고 제목·축 이름 붙이기

예상 결과

설비별 평균 온도 막대(C호기 최고), 색·가로 변형

posco

실습 1. 막대 그래프 직접 그리기 정답

설비별 평균 온도를 막대로, 색·가로 막대로 변형

CODE · 실습 1

```
평균 = df.groupby('설비명')['온도'].mean()
plt.bar(평균.index, 평균.values)
plt.bar(평균.index, 평균.values, color=['blue','blue','red'], width=0.5)
plt.barh(평균.index, 평균.values)          # 가로 막대
plt.title('설비별 평균 온도'); plt.ylabel('온도(℃)')
plt.show()
```

posco

실습 2. 범주별 집계 막대 그래프

개수·평균을 집계해 범주별로 막대 비교

목표

value_counts·groupby로 집계한 값을 막대로 비교

단계

· 상태 열의 개수를 세어 막대로 그리기

· 구역으로 그룹을 나눠 평균 진동을 막대로 그리기

· 제목·축 이름을 붙여 완성

예상 결과

상태 정상 17·점검 4, 구역별 평균 진동 비교

posco

실습 2. 범주별 집계 막대 그래프 정답

개수·평균을 집계해 범주별로 막대 비교

CODE · 실습 2

```
상태 = df['상태'].value_counts()
plt.bar(상태.index, 상태.values)           # 정상 17, 점검 4
구역 = df.groupby('구역')['진동'].mean()
plt.bar(구역.index, 구역.values)           # 1구역 2.33, 2구역 3.98
plt.title('구역별 평균 진동'); plt.ylabel('진동(mm/s)')
plt.show()
```

02

히스토그램 — 분포 확인

숫자가 어떤 구간에 얼마나 몰려 있는지 분포 읽기

posco

막대와 히스토그램의 차이

생긴 모양은 비슷해도 목적이 완전히 다른 두 그래프

막대 그래프

정해진 항목별 크기 비교

A호기·B호기처럼 미리 정해진 항목의 값을 막대 높이로 비교 — 가로축은 항목 이름

입력 — 항목·값 두 개 · 명령 — bar

히스토그램

숫자가 퍼진 분포 확인

숫자값이 어떤 구간에 얼마나 몰려 있는지 분포를 봄 — 가로축은 자동으로 나뉜 숫자 구간

입력 — 숫자 목록 하나 · 명령 — hist

posco

plt.hist()로 히스토그램 그리기

숫자 목록 하나만 넣으면 구간별 개수 자동 계산

CODE

```
plt.hist(온도목록)
plt.xlabel('온도(℃)')
plt.ylabel('설비 수')
plt.show()
```

세로축은 개수 — 막대가 높다는 건 그 구간에 값이 많다는 뜻 (막대 그래프와 다름)

posco

bins로 구간 나누기

히스토그램에서 가장 중요한 옵션, 데이터를 몇 개 구간으로

BINS · 구간 수

bins=10이면 10개 구간

bins=20이면 잘게, bins=5면 크게 — 적게 나누면 큰 모양만, 많이 나누면 누면 세밀한 모양까지 보임

plt.hist(데이터, bins=10)

조절 원칙

분포 모양은 bins에 따라 달라짐

정해진 정답 없음 — 데이터가 많으면 구간 많게, 적으면 적게가 보통, 직접 바꿔보며 가장 잘 보이는 값 찾기

데이터 20개에 bins 30이면 빈 구간 많음

posco

bins를 바꿔보며 분포 관찰하기

여러 bins로 그려보고 일관되게 나타나는 특징만 신뢰

잘못된 설정

너무 작거나 너무 크면

3처럼 너무 작으면 65 근처인지 69 근처인지 세부가 사라짐 — 50처럼 너무 크면 막대가 듬성듬성해 분포가 안 보임

한 가지 설정만 믿으면 우연한 모양에 속음

출발점

데이터 개수의 √(제곱근)

100개면 10개 정도가 출발점 — 그리고 직접 그려보며 어느 설정에서도 일관된 특징만 채택

직접 그려보며 고르는 게 최선

히스토그램으로 분포 읽기

그릴 줄 아는 것과 읽을 줄 아는 것은 다른 일

01 · 봉우리 위치

정상 작동 범위

값이 가장 많이 몰린 구간 — 봉우리가 65도면 대부분 65도 근처에서 작동

대표값 · 보통의 상태

02 · 퍼진 정도

설비 간 편차 크기

좁은 범위에 모여 있으면 고른 상태, 넓게 퍼져 있으면 편차 큼

잘 관리되는 설비 — 좁게 모임

03 · 동떨어진 값

이상 후보 설비

봉우리에서 멀리 떨어진 작은 막대 — 점검이 필요한 설비를 가리킴

이상 탐지의 기초 — 정상 모양을 알아야 이상을 찾음

히스토그램 해석 프레임

분포를 볼 때마다 따라가는 세 단계 흐름

STEP 1 · 중심

어디 모였나

값들이 주로 어디 모여 있나, 봉우리가 어디인지 — 대표값

≈ 평균

STEP 2 · 퍼짐

얼마나 흩어졌나

중심 주변에 모였나 넓게 흩어졌나 — 데이터의 안정성

≈ 표준편차

STEP 3 · 이상

동떨어진 값

중심에서 멀리 떨어져 따로 노는 값 — 점검 대상을 찾는 핵심

이상 탐지 핵심

예) 봉우리 65도 정상 기준, 60~70 좁은 범위면 관리 양호, 88도 작은 막대가 점검 대상

실습 3. 히스토그램 직접 그리기

온도 분포를 히스토그램으로, bins로 구간 조절

목표

숫자 분포를 히스토그램으로 그리고 bins로 구간 조절

단계

· 온도 열을 hist로 그려 자동 구간 분포 확인

· bins를 작게·크게 바꿔 구간 수에 따른 모양 비교

· 제목·축 이름을 붙여 완성

예상 결과

온도 구간별 개수 분포, bins 5·20의 모양 차이

posco

실습 3. 히스토그램 직접 그리기 정답

온도 분포를 히스토그램으로, bins로 구간 조절

CODE · 실습 3

```
plt.hist(df['온도'])              # 자동 구간
plt.hist(df['온도'], bins=5)      # 굵게
plt.hist(df['온도'], bins=20)     # 잘게
plt.title('온도 분포'); plt.xlabel('온도(℃)'); plt.ylabel('개수')
plt.show()
```

실습 4. 센서값 분포 히스토그램

분포를 그리고 평균·표준편차·이상 설비로 해석

목표

분포를 그리고 평균·표준편차와 이상 설비로 해석

단계

· 온도 분포를 히스토그램으로 그리기

· 평균과 표준편차를 구해 분포의 중심·퍼짐과 비교

· 조건 필터링으로 정상 범위를 벗어난 설비 수 확인

예상 결과

온도 평균 70.0·표준편차 5.6, 78 이상 설비 2대

posco

실습 4. 센서값 분포 히스토그램 정답

분포를 그리고 평균·표준편차·이상 설비로 해석

CODE · 실습 4

```
plt.hist(df['온도'], bins=8)
print(round(df['온도'].mean(), 1), round(df['온도'].std(), 1))   # 70.0 5.6
높은온도 = df[df['온도'] >= 78]
print(len(높은온도))              # 2 (정상 범위 벗어난 설비)
plt.show()
```

03

산점도 — 두 값의 관계

점들의 모양으로 양·음·무관계와 패턴 읽기

posco

두 변수의 관계를 보고 싶을 때

두 값을 가로·세로에 두고 점을 찍어 흩어진 모양으로 관계 읽기

선 그래프

값 하나의 변화

시간에 따른 온도 하나의 변화를 봄

plt.plot()

히스토그램

값 하나의 분포

온도 하나가 어떻게 퍼져 있는지 분포를 봄

plt.hist()

산점도

두 값의 관계

한 대마다 온도 가로, 진동 세로에 점을 찍어 100대면 100개 점

plt.scatter()

오른쪽 위로 줄지으면 함께 큼, 마구 흩어지면 별 관계 없음

posco

plt.scatter()로 산점도 그리기

점만 찍고 잇지 않음 — 선 그래프와 결정적 차이

CODE

```
plt.scatter(온도, 진동)
plt.xlabel('온도(℃)')
plt.ylabel('진동(mm/s)')
plt.show()
```

산점도는 순서가 중요하지 않음 — 보는 건 점들의 흩어진 전체 모양

점 색·크기·투명도 옵션

점이 많아 겹칠 때 — alpha로 밀도까지 드러내기

옵션	역할	예시
color	점 색 — 정상 파랑, 점검 빨강으로 그룹 구분	**'blue'**
s	점 크기 — 작아 안 보이면 키우고, 뭉치면 줄이기	
alpha	투명도 — 0에 가까우면 투명, 1에 가까우면 진함	**0.5**

alpha=0.3으로 낮추면 많이 겹친 곳은 진하게, 적게 겹친 곳은 연하게 — 밀도가 드러남

산점도로 상관 패턴 읽기

점들의 모양은 크게 세 가지 패턴

점들의 모양	관계	의미
오른위로 줄짐	**양의 관계**	함께 커짐 — 온도 높을수록 진동도 큰 경향
오른아래 줄짐	**음의 관계**	하나 크면 하나 작음 — 관리 점수 높을수록 고장 적음
둥글게 흩어짐	**관계 없음**	따로 움직임 — 한 값이 변해도 다른 값은 안 따라감

상관과 인과는 다름 — 함께 변한다고 한쪽이 원인은 아님 (아이스크림과 익사, 둘 다 여름 때문)

산점도 위에 기준선 그리기 (axhline)

가로 기준선으로 **임계값 표시**

CODE

```
plt.scatter(온도, 진동)
plt.axhline(y=4.0, color='red',
 linestyle='--')
plt.show()
```

ax=축, h=가로, line=선 — 세로 기준선은 axvline (v=vertical)

posco

4가지 그래프 선택 기준

핵심은 데이터 생김새가 아니라 내가 무엇을 알고 싶은가

보고 싶은 것	그래프	질문 예시
시간 변화	**선 그래프**	온도가 어떻게 변했나
항목 비교	**막대 그래프**	어느 설비가 큰가
값의 분포	**히스토그램**	어디에 몰렸나
두 값 관계	**산점도**	함께 변하나

같은 온도 데이터도 질문에 따라 그래프가 달라짐

posco

실습 5. 산점도로 센서 간 관계 분석

온도-진동 산점도에 상태별 색·기준선 추가

목표

두 변수 산점도에 상태별 색과 임계 기준선 추가

단계

· 온도와 진동을 산점도로 찍어 관계 확인

· 상태를 색으로 매핑해 점검·정상을 색으로 구분

· axhline으로 진동 임계 기준선 그리기

예상 결과

온도↑ 진동↑ 양의 관계, 상태별 색·기준선 표시

posco

실습 5. 산점도로 센서 간 관계 분석 정답

온도-진동 산점도에 상태별 색·기준선 추가

CODE · 실습 5

```
plt.scatter(df['온도'], df['진동'])
색 = df['상태'].map({'정상':'blue', '점검':'red'})
plt.scatter(df['온도'], df['진동'], color=색, alpha=0.6)   # 상태별 색
plt.axhline(y=4.0, color='red', linestyle='--')          # 진동 기준선
plt.title('온도와 진동의 관계'); plt.xlabel('온도(℃)'); plt.ylabel('진동(mm/s)')
plt.show()
```

posco

실습 6. 두 변수 관계 자유 분석

스스로 고른 두 센서의 관계를 산점도로 분석

목표

스스로 고른 두 센서의 관계를 산점도로 분석

단계

· 관심 있는 두 센서 열을 고르기

· 산점도로 두 값의 관계를 점으로 확인

· 제목·축 이름을 붙이고 관계 방향을 해석

예상 결과

온도와 소음의 관계 산점도(함께 오르는지 확인)

posco

실습 6. 두 변수 관계 자유 분석 정답

스스로 고른 두 센서의 관계를 산점도로 분석

CODE · 실습 6

```
plt.scatter(df['온도'], df['소음'], alpha=0.6)
plt.title('온도와 소음의 관계')
plt.xlabel('온도(℃)'); plt.ylabel('소음(dB)')
plt.show()
```

posco

04

시각화 리포트 완성

subplot으로 네 그래프를 엮어 하나의 분석 리포트로

posco

한 화면에 여러 그래프 (subplot)

subplot(행, 열, 칸번호), 칸 번호는 1부터

CODE

```
plt.subplot(1, 2, 1)
plt.plot(...)
plt.subplot(1, 2, 2)
plt.bar(...)
plt.show()
```

칸 번호는 책 읽는 순서 — 왼쪽에서 오른쪽, 위에서 아래로 (1부터 시작)

posco

subplot 배치와 각 그래프 제목

각 칸엔 title, 전체엔 suptitle, 겹침은 tight_layout

명령	역할
`plt.title()`	각 칸 제목 — 칸 선택 후 호출, 축 이름도 칸별로 붙음
`plt.suptitle()`	전체 화면 제목 — sup은 전체를 뜻함
`plt.tight_layout()`	칸 겹침 정리 — show 직전 한 줄, 거의 습관처럼 함께

작은 한 줄이지만 여러 그래프를 다룰 때 없어선 안 되는 명령

posco

시각화 리포트 구성 흐름

그래프에도 이야기 순서가 있다 — 큰 그림에서 세부로

STEP 1 · 전체현황

막대

데이터의 전체 모습 — 범주별 개수나 비중

큰 그림부터

→

STEP 2 · 분포

히스토그램

주요 값들이 어떻게 퍼졌는지 — 정상 모양과 이상값

분포 확인

→

STEP 3 · 관계

산점도

값들 사이 관계 — 숨은 연결을 드러냄

관계 분석

→

STEP 4 · 결론

정리

발견을 한두 문장으로 — C호기가 온도·진동 모두 높아 점검 필요처럼

리포트의 마침표

사람은 큰 그림에서 세부로 좁혀가며 이해함 — 전체부터 차근차근

posco

전처리 결과를 시각화로 검증하기

시각화는 분석 전 과정의 눈 — 처리 전후를 그래프로 검증

STEP 1
불러오기
데이터를 파일에서 읽어 옴
read_csv

→

STEP 2
구조 확인
몇 행 몇 열, 어떤 자료형인지 파악
info

→

STEP 3
전처리
결측치는 평균으로 채우고, 이상치는 제거
결측·이상치

→

STEP 4
시각화 검증
채우기·제거 전후 분포를 그래프로 눈 확인
그래프

→

STEP 5
리포트
네 그래프 + 결론으로 분석 리포트 완성
완성

전처리는 한 번 하고 끝이 아니라 결과를 확인하고 조정하는 반복 — 그 확인에서 그래프가 결정적 역할

실습 7. SECOM 불러오기와 기초 확인

SECOM을 불러와 크기·결측을 확인하고 처리

목표

실제 공정 데이터를 불러와 크기·결측을 확인하고 처리

단계

· SECOM 데이터를 불러와 크기와 전체 결측 개수 확인

· 평균으로 결측을 채워 처리

· 처리 후 결측이 0인지 검증

예상 결과

SECOM 120행 23열·결측 98개 → 처리 후 0

posco

실습 7. SECOM 불러오기와 기초 확인 정답

SECOM을 불러와 크기·결측을 확인하고 처리

CODE · 실습 7

```
secom = pd.read_csv('secom_샘플.csv')
print(secom.shape, int(secom.isna().sum().sum()))   # (120, 23) 98
secom2 = secom.fillna(secom.mean(numeric_only=True))
print(int(secom2.isna().sum().sum()))                # 0
```

posco

실습 8. 시각화 리포트 완성과 저장

분포·범주·관계를 2×2 한 화면에 모아 저장

목표

분포·범주·관계 그래프를 2×2로 모아 리포트로 저장

단계

· figure와 subplot으로 2×2 칸을 만들기

· 각 칸에 분포·범주·관계·판정 그래프를 그리고 제목 달기

· tight_layout으로 정리하고 이미지 파일로 저장

예상 결과

센서 분포·라인 개수·관계·판정을 담은 2×2 리포트 저장

posco

실습 8. 시각화 리포트 완성과 저장 정답

분포·범주·관계를 2×2 한 화면에 모아 저장

CODE · 실습 8

```
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1); plt.hist(secom2['sensor_01'], bins=15); plt.title('센서01 분포')
plt.subplot(2, 2, 2); vc = secom2['line'].value_counts()
plt.bar(vc.index, vc.values); plt.title('라인별 개수')
plt.subplot(2, 2, 3); plt.scatter(secom2['sensor_01'], secom2['sensor_02'], alpha=0.5)
plt.subplot(2, 2, 4); rc = secom2['result'].value_counts()
plt.bar(rc.index, rc.values); plt.title('판정 결과')
plt.tight_layout(); plt.savefig('시각화리포트.png')
```